

GraphRAG with LlamaIndex + Neo4j + Ollama (Gemma3:12B)^[6pt]

A simple, hands-on guide tailored for ATC transcripts

Who this is for. You want to **query ATC transcripts** and related entities (callsigns, runways, intents) using a **graph database (Neo4j)** plus an **LLM (Gemma3:12B via Ollama)**. You already ingest or will ingest data into Neo4j; you want LlamaIndex to *read and query* that graph. This guide uses very simple language and runnable code. It follows the LlamaIndex property-graph patterns and retrievers — as shown in their docs .

Contents

1	What You Will Build (High-Level)	1
2	Prerequisites	2
2.1	Install Software	2
2.2	Create a Clean Environment (Optional)	2
2.3	Install Python Packages	2
3	Start Neo4j and Set Environment Variables	2
3.1	Run Neo4j via Docker (Quick Start)	2
3.2	Environment Variables	3
4	Test the Connection to Neo4j	3
5	Your ATC Graph Schema (Simple, Strict)	3
5.1	Our ATC Types	3
5.2	Schema in Code (Validation Map)	4
6	Configure the Local LLM (Ollama, Gemma3:12B)	4
7	Load Your <i>Existing</i> Graph into a PropertyGraphIndex	4
8	Hybrid Retrieval (Optional but Powerful)	5
8.1	Add a Small, Fast Embedding Model	5
8.2	Two Useful Retrievers	5
9	Query Engine (Ask Questions, Get Answers)	6
10	Direct Cypher (When You Want Full Control)	6
11	End-to-End: Minimal Starter Script	7
12	How This Fits Your 3-Agent Workflow	8
13	Troubleshooting Tips	8
14	Next Steps	8

1. What You Will Build (High-Level)

- A **Neo4j graph** with nodes like `Transcript`, `Entity`, `Category`, `IntentNode`, and edges like `CONTAINS`, `ISA`, `HASINTENT`.

- A LlamaIndex PropertyGraphIndex that *connects to your existing Neo4j* and lets you query with:
 - Natural language → Cypher (Text-to-Cypher retriever)
 - Vector similarity (hybrid retrieval)
- Ollama (Gemma3:12B) as the local LLM to help interpret questions and generate Cypher, fully local.

In LlamaIndex terms, this is a **Property Graph Index** used for **construction and querying** of labeled nodes and relations .

2. Prerequisites

2.1. Install Software

- **Python** 3.10+ (recommended)
- **Neo4j** (via Docker or local install)
- **Ollama** running locally (port 11434)
- **Pip** packages (see below)

2.2. Create a Clean Environment (Optional)

```
# Example: create a fresh venv
python -m venv ~/venvs/graphrag
source ~/venvs/graphrag/bin/activate
python -m pip install --upgrade pip
```

2.3. Install Python Packages

LlamaIndex components for Neo4j, Ollama, and embeddings (HuggingFace) — the same families used in the docs .

```
pip install
llama-index
llama-index-graph-stores-neo4j
llama-index-llms-ollama
llama-index-embeddings-huggingface
neo4j
```

3. Start Neo4j and Set Environment Variables

3.1. Run Neo4j via Docker (Quick Start)

This mirrors the documentation’s approach to launching Neo4j locally .

```
docker run
-p 7474:7474 -p 7687:7687
-v $PWD/data:/data -v $PWD/plugins:/plugins
--name neo4j-apoc
-e NEO4J_apoc_export_file_enabled=true
```

```
-e NEO4J_apoc_import_file_enabled=true
-e NEO4J_apoc_import_file_use__neo4j__config=true
-e NEO4JLABS_PLUGINS='["apoc"]'
neo4j:latest
```

Open `[http://localhost:7474]`(`http://localhost:7474`), log in with `neo4j` / `neo4j`, and set a new password.

3.2. Environment Variables

Later code snippets assume these exist (this follows the pattern used in examples) .

```
export NEO4J_URI="bolt://localhost:7687"
export NEO4J_USER="neo4j"
export NEO4J_PASSWORD="YOUR_PASSWORD"

# Optional: if you want to set defaults for Ollama model

export OLLAMA_BASE_URL="[http://localhost:11434] (http://localhost:11434)"
export OLLAMA_MODEL="gemma3:12b"
```

4. Test the Connection to Neo4j

```
from llama_index.graph_stores.neo4j import Neo4jPropertyGraphStore

graph_store = Neo4jPropertyGraphStore(
    username="neo4j",
    password="YOUR_PASSWORD",
    url="bolt://localhost:7687",
)

# Try a simple schema fetch (string for human-readability)

schema_str = graph_store.get_schema_str()
print("Connected. Current schema:\n", schema_str)
```

If this prints something (even if minimal), your connection works.

5. Your ATC Graph Schema (Simple, Strict)

LlamaIndex supports defining a **strict schema** for graph extraction/query wiring (entities, relations, and what is allowed) — this helps avoid nonsense relations, as recommended in the docs .

5.1. Our ATC Types

- Nodes:
 - `Transcript(text, speaker)`
 - `Entity(name, type)` (e.g., `CALLSIGN`, `RUNWAY`, `TAXIWAY`)
 - `Category(name)` (e.g., `AIRLINE`, `NUMBER`, `PHONETIC`)
 - `IntentNode(label)` (e.g., `REQUESTTAXI`, `HOLDSHORT`)
- Relations:

- (Transcript)-[:CONTAINS]->(Entity)
- (Entity)-[:IS_A]- > (Category)
- (Transcript)-[:HAS_INTENT]- > (IntentNode)

5.2. Schema in Code (Validation Map)

This mirrors the pattern shown for predefined schemas (entities, relations, validation) in the docs .

```
from typing import Literal

# 1) Node "labels" we care about

Entities = Literal["TRANSCRIPT", "ENTITY", "CATEGORY", "INTENTNODE"]

# 2) Relation "types" we allow

Relations = Literal["CONTAINS", "IS_A", "HAS_INTENT"]

# 3) Validation: who can connect to what (STRICT)

validation_schema = {
    "TRANSCRIPT": ["CONTAINS", "HAS_INTENT"],
    "ENTITY": ["IS_A"],
    "CATEGORY": [],
    "INTENTNODE": [],
}
```

Why this matters (in simple words). It prevents illegal links (e.g.,

{ENTITY -[:HAS_INTENT]-> CATEGORY}). LlamaIndex retrievers will use the schema information to improve query generation and safety .

6. Configure the Local LLM (Ollama, Gemma3:12B)

The docs show examples of using `Ollama(...)` as a local LLM provider. We do the same, but with `gemma3:12b` and your local base URL .

```
from llama_index.llms.ollama import Ollama
from llama_index.core import Settings

llm = Ollama(
    model="gemma3:12b",
    base_url="http://localhost:11434 (http://localhost:11434)",
    request_timeout=3600, # long timeout for big prompts
    # context_window can be set to limit memory if needed
    # context_window=8000,
)
Settings.llm = llm
```

7. Load Your *Existing* Graph into a PropertyGraphIndex

In your workflow, ingestion already happened (Phase 2). We now *wrap* that graph with LlamaIndex for retrieval. This pattern is straight from the docs.

```

from llama_index.core import PropertyGraphIndex

index = PropertyGraphIndex.from_existing(
    property_graph_store=graph_store,
    # Optional: add a vector store or embed_kg_nodes if using hybrid search
    # vector_store=...,
    # embed_kg_nodes=True,
)

```

8. Hybrid Retrieval (Optional but Powerful)

The documentation shows combining

```

{vector retrieval} with {graph/Cypher retrieval} to get better results (e.g.,
    synonyms + structure) .

```

8.1. Add a Small, Fast Embedding Model

```

from llama_index.embeddings.huggingface import HuggingFaceEmbedding

embed_model = HuggingFaceEmbedding(model_name="BAAI/bge-small-en-v1.5")

```

8.2. Two Useful Retrievers

- **TextToCypherRetriever**: convert natural language → Cypher using the schema LLM.
- **VectorContextRetriever**: find relevant nodes by vector similarity and expand nearby paths.

```

# Both are shown in the docs and can be combined in one as_retriever() call .
from llama_index.core.indices.property_graph import (
    TextToCypherRetriever,
    VectorContextRetriever,
)

cypher_retriever = TextToCypherRetriever(
    index.property_graph_store,
    llm=llm, # our local Gemma3:12b via Ollama
)

vector_retriever = VectorContextRetriever(
    index.property_graph_store,
    embed_model=embed_model,
    include_text=False, # set True to return source text chunks too
    similarity_top_k=4, # tune for how many nodes to pull
    path_depth=1, # how far to expand neighbors
)

retriever = index.as_retriever(
    sub_retrievers=[cypher_retriever, vector_retriever]
)

```

9. Query Engine (Ask Questions, Get Answers)

```
# The docs show that you can directly make a query_engine from your index or from your
# custom retrievers .
from llama_index.core.query_engine import RetrieverQueryEngine

query_engine = RetrieverQueryEngine.from_args(
    retriever, # the combined retriever we built
    llm=llm, # local LLM for synthesis and explanations
)

# Example 1: Which transcripts mention "whiskey" and what is the intent?

resp = query_engine.query(
    "List transcripts that mention 'whiskey' and show their HAS_INTENT."
)
print(resp)

# Example 2: Find all callsigns that appear in pilot utterances (speaker='pilot')

resp = query_engine.query(
    "Show callsigns mentioned by pilot speakers only."
)
print(resp)

# Example 3: Given this text, what intent node is most likely?

sample = "ifly one zero zero three, taxi whiskey hold short victor seven"
resp = query_engine.query(
    f"Find the most likely IntentNode for: {sample}"
)
print(resp)
```

How it works (simple). The engine tries vector lookup *and* text-to-Cypher. The LLM then explains/aggregates the results into a readable answer. This pattern is exactly what the documentation recommends for property-graph querying with multiple retrievers .

10. Direct Cypher (When You Want Full Control)

Sometimes you know exactly what you want. You can execute Cypher directly through the graph store (the docs demonstrate both programmatic and templated-Cypher routes) .

```
cypher = """
MATCH (t:Transcript)-[:CONTAINS]->(e:Entity)
WHERE t.speaker = 'pilot' AND e.type = 'CALLSIGN'
RETURN t.text AS transcript, e.name AS callsign
LIMIT 10
"""
rows = graph_store.structured_query(cypher)
for r in rows:
    print(r)
```

Safety tip: When using LLM-generated Cypher in production, prefer read-only roles / sand-boxes (the docs call this out) .

11. End-to-End: Minimal Starter Script

This puts it all together for quick testing.

```
# file: run_graphrag_atc.py

from llama_index.graph_stores.neo4j import Neo4jPropertyGraphStore
from llama_index.core import PropertyGraphIndex, Settings
from llama_index.llms.ollama import Ollama
from llama_index.embeddings.huggingface import HuggingFaceEmbedding
from llama_index.core.indices.property_graph import (
    TextToCypherRetriever,
    VectorContextRetriever,
)
from llama_index.core.query_engine import RetrieverQueryEngine

# 1) Connect to Neo4j

graph_store = Neo4jPropertyGraphStore(
    username="neo4j",
    password="YOUR_PASSWORD",
    url="bolt://localhost:7687",
)

# 2) Local LLM (Gemma3:12b via Ollama)

llm = Ollama(model="gemma3:12b", base_url="[http://localhost:11434] (http://localhost:11434)")
Settings.llm = llm

# 3) Wrap your existing graph

index = PropertyGraphIndex.from_existing(property_graph_store=graph_store)

# 4) Optional: vector embeddings for hybrid search

embed_model = HuggingFaceEmbedding(model_name="BAAI/bge-small-en-v1.5")

cypher_retriever = TextToCypherRetriever(index.property_graph_store, llm=llm)
vector_retriever = VectorContextRetriever(
    index.property_graph_store, embed_model=embed_model,
    include_text=False, similarity_top_k=4, path_depth=1
)

retriever = index.as_retriever(
    sub_retrievers=[cypher_retriever, vector_retriever]
)
query_engine = RetrieverQueryEngine.from_args(retriever, llm=llm)

# 5) Ask questions

print(query_engine.query(
    "Show transcripts where the intent is taxi or hold short, with callsigns."
))
```

12. How This Fits Your 3-Agent Workflow

The docs make it easy to plug different retrieval pieces together as reusable modules (retrievers, query engine). You can map that to your pipeline like this :

- **Agent 1: Segmenter** — Use `VectorContextRetriever` to fetch known lexicon nodes (callsigns, runways) and compare them against the raw text. If confidence is low, ask the LLM for synonyms or likely matches.
- **Agent 2: Grounder** — Use `TextToCypherRetriever` or direct Cypher to **verify** claims (*Does this transcript contain a valid callsign? What linked intent exists?*).
- **Agent 3: Predictor** — Ask the LLM (with graph context) to produce final (**Speaker, Intent, Tags**) using both the transcript text and retrieved graph neighbors.

Because LlamaIndex lets you **combine retrievers**, you can easily compose the above into one `RetrieverQueryEngine` and swap components as you iterate .

13. Troubleshooting Tips

- **Neo4j not reachable?** Check container is running and `bolt://localhost:7687` is correct. Try fetching `graph_store.get_schema_str()`.
- **Ollama port in use (11434)?** Stop any prior server using that port, or change `base_url`. On macOS: `lsof -i :11434`.
- **Text-to-Cypher seems wrong?** Start with strict schema and short questions. You can also sanitize/validate generated Cypher (docs mention a validator hook) .
- **Slow answers?** Use a smaller embedding model, reduce `similarity_top_k`, or `limit_path_depth`. Consider
- **Security (important).** In production, run with read-only DB roles when allowing LLM-generated Cypher, as the docs advise .

14. Next Steps

- Add **metadata filters** (`speaker=pilot, airport=WSSS`) to narrow results, as shown in the vector/filter examples in the docs .
- Store **source text nodes** and connect them to entities for better explanations (`HAS_SOURCE_patterns_appear`
- **Build evaluation scripts:** Given a question and expected Cypher, test whether retrievers generate the correct query and whether answers match ground truth.

Primary reference used while writing this guide: LlamaIndex property-graph documentation and examples for Neo4j (construction, retrievers, query engine, and Ollama usage) — see the uploaded PDF. Source: